

Algoritmos de Busca

Do problema ao espaço de busca

Antes de escolher um algoritmo, precisamos representar onde ele irá buscar.



Prof. Dr. João Choma

Universidade Estadual de Maringá — UEM

github.com/JoaoChoma

Representação do Espaço de Busca



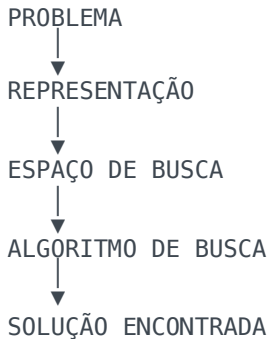
O que significa buscar?

Considere um problema:

Existe uma situação inicial e queremos encontrar uma sequência de decisões que nos leve a uma situação desejada.

Em um problema de busca, precisamos identificar:

- **Problema** — o que precisa ser resolvido?
- **Solução buscada** — o que caracteriza uma resposta válida?
- **Espaço de busca** — onde estão as possibilidades?
- **Algoritmo de busca** — como exploraremos essas possibilidades?





O algoritmo não busca diretamente no problema real.

*Ele busca em uma **representação computacional do problema**.*

Exemplo: encontre a saída



	0	1	2	3	4
0	S			■	
1	■	■		■	
2					
3		■	■	■	
4					G



Exemplo: encontre a saída

S = início **G** = objetivo ■ = obstáculo

Pergunta

Como transformar esse labirinto em algo que um algoritmo consiga explorar?



Não queremos apenas "chegar ao G".
que transforme o estado inicial no estado
objetivo.

Portanto:

Dependendo do problema, podemos procurar:

- a solução com menos passos;
- a melhor solução segundo algum critério.

Queremos encontrar uma sequência de ações:
 $S \rightarrow \text{direita} \rightarrow \text{direita} \rightarrow \text{baixo} \rightarrow$
 $\text{baixo} \rightarrow \dots$

***Solução = caminho ou sequência
de ações que satisfaz o objetivo.***

- qualquer solução;
- a solução de menor custo;



Representando um estado

Cada posição possível pode ser representada por:
(linha, coluna)



Exemplos:

$(0,0)$ $\leftarrow$ início

$(0,1)$

$(0,2)$

$(1,2)$

...

$(4,4)$ $\leftarrow$ objetivo



Um **estado** descreve uma situação relevante do problema.



Estado inicial

$S = (0, 0)$ Estado objetivo

$G = (4, 4)$ Podemos representar o teste de objetivo como:

```
def objetivo(estado):
```

```
    return estado == (4, 4) O algoritmo precisa ser capaz de responder:
```

"Este estado resolve o problema?"



Para cada estado podemos definir operadores:

↑
subir
← esquerda (x,y) direita →
descer
↓

Uma ação só é válida quando:

- permanece dentro dos limites;
- não entra em um obstáculo;
- respeita as regras do problema.



Uma ação transforma um estado em outro:

ESTADO + AÇÃO $\rightarrow$ NOVO ESTADO Exemplo:

$(0,1)$ + direita $\rightarrow (0,2)$ Outro exemplo:

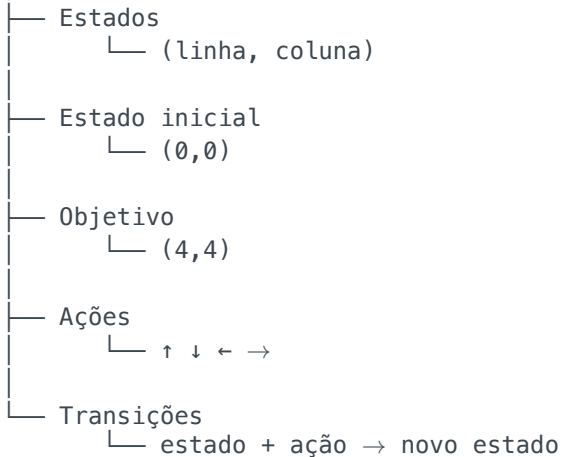
$(0,2)$ + baixo $\rightarrow (1,2)$ Mas:

$(0,2)$ + direita $\rightarrow$ BLOQUEADO porque $(0,3)$ contém um obstáculo.



Temos agora uma representação do problema

Problema



Temos agora uma representação do problema



Agora podemos construir o **espaço de busca**.



O que é o espaço de busca?

O espaço de busca representa:

todas as situações que podem ser alcançadas e as transições possíveis entre elas.

No labirinto:

- cada posição válida → **estado**
- cada movimento válido → **transição**

Assim, o labirinto pode ser representado como um **grafo**.

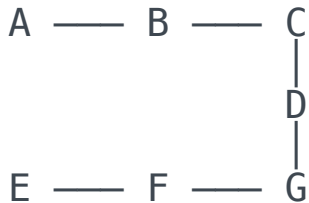


Considere uma versão menor:

A	B	C
■	■	D
E	F	G



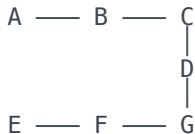
As conexões possíveis são:





Do labirinto para um grafo

O problema visual tornou-se um **espaço de estados conectado por transições**.



Esse grafo responde:

Onde o algoritmo pode procurar?

Ele ainda não diz:

Como o algoritmo irá procurar?

Essa é responsabilidade da estratégia de busca.



Espaço de busca

Algoritmo de busca

Define **como explorar as possibilidades**.

BFS? DFS? UCS? Greedy? A*?

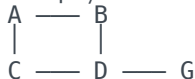
*O mesmo espaço pode ser explorado
por diferentes algoritmos.*

Representa **o que é possível**.



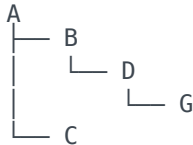


O espaço de estados pode conter ciclos:



- **Espaço de estados:** estrutura do problema.
- **Árvore de busca:** exploração realizada pelo algoritmo.

Durante a execução, o algoritmo gera uma árvore de busca:





O espaço precisa ser construído inteiro?

Não necessariamente.

Em muitos problemas, o espaço é grande demais para ser materializado.



Podemos gerar os próximos estados **sob demanda**:

Estado atual



Gerar ações possíveis



Gerar estados sucessores



Adicionar à fronteira

O espaço precisa ser construído inteiro?



Esse processo acontece durante a busca.



Considere:



Se já exploramos S, temos:

Visitados: S

Fronteira: A, B, C A **fronteira**

contém estados descobertos que ainda podem ser explorados.



Temos:

Fronteira = A, B, C Qual estado devemos explorar agora?

A?

B?

C?

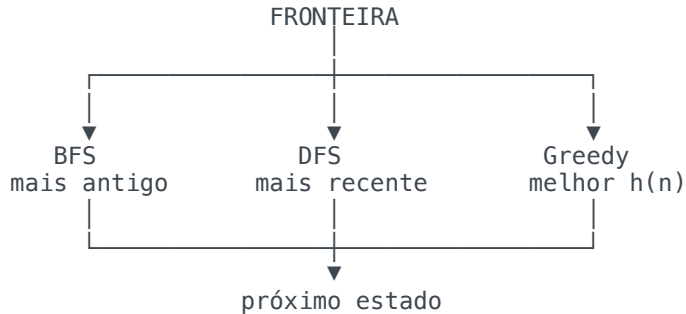
*A resposta depende do **algoritmo de busca**.*



```
fronteira = [estado_inicial]
visitados = set()
while fronteira:
    estado = selecionar(fronteira)
    if objetivo(estado):
        return solucao(estado)
    visitados.add(estado)
    for sucessor in gerar_sucessores(estado):
        if sucessor not in visitados:
            fronteira.append(sucessor)
```



A função crítica é:
`selecionar(frenteira)`





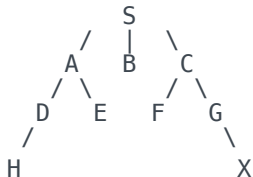
Mesmo problema, estratégias diferentes

O espaço de busca continua sendo o mesmo.

O que muda é a **estratégia para escolher onde buscar**.



Considere:



X é o objetivo.

A BFS explora por **níveis**.



Nível 0:

Nível 1:

Nível 2:

Nível 3:

		S	
	A	B	C
D	E	F	G
H			X

Possível ordem:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow$

$H \rightarrow X$ Estrutura utilizada:

FILA --- FIFO

A BFS prioriza estados menos profundos.



No mesmo espaço:



X A DFS escolhe um ramo e tenta avançar o máximo possível.



Uma possível exploração:

S
|
A
|
D
|
H

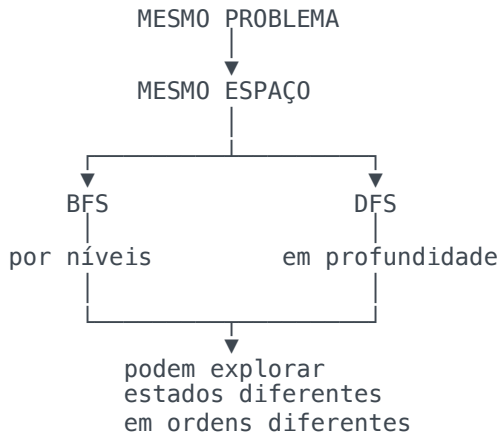
Sem solução?

volta... Depois continua explorando outros ramos.

Estrutura típica:

PILHA --- LIFO

O que aprendemos com BFS e DFS?



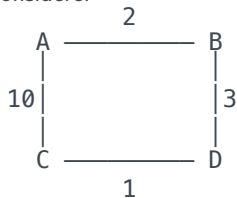


O algoritmo determina **como o espaço é explorado**.

E se as ações tiverem custos?



Agora considere:





E se as ações tiverem custos?

Uma transição passa a ter:

estado $\rightarrow$ ação $\rightarrow$ novo estado
+

custo Agora podemos perguntar:

*Qual é o caminho de **menor custo**?*



Definimos:

$g(n)$ = custo acumulado desde o estado inicial até n Exemplo:

$S \xrightarrow{2} A \xrightarrow{4} B$

$g(S) = 0$

$g(A) = 2$

$g(B) = 6$ Algoritmos como **Uniform Cost Search** priorizam o menor $g(n)$.



Imagine:

S	.	.	.	.
.	■	■	.	.
.	.	.	.	G



Alguns estados parecem estar mais próximos de G.
Podemos estimar essa distância usando uma **heurística**.

$h(n)$ = estimativa do custo de n até o objetivo



Para uma grade, podemos usar a distância Manhattan:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

Estado atual: (1,2)

Objetivo: (4,4)

$$h(n) = |1-4| + |2-4|$$

$$= 3 + 2$$

= 5 A heurística ajuda o algoritmo a decidir **onde parece mais promissor buscar**.



Algoritmo	Prioriza
BFS	menor profundidade
DFS	maior profundidade / último descoberto
Uniform Cost	menor $g(n)$
Greedy	menor $h(n)$
A*	menor $g(n) + h(n)$

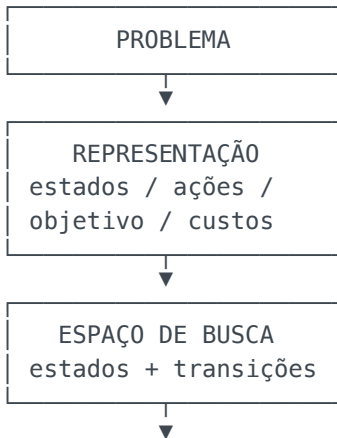
No A*:

$$f(n) = g(n) + h(n)$$

- $g(n) \rightarrow$ quanto já custou;
- $h(n) \rightarrow$ quanto estimamos faltar.



O ciclo completo





ALGORITMO DE BUSCA
estratégia de
exploração



SOLUÇÃO ENCONTRADA
caminho / ações /
estado objetivo



Problema dos recipientes

Temos:

- um recipiente de **5 litros**;
- um recipiente de **3 litros**;
- água disponível;
- nenhuma marcação intermediária.

Objetivo

Obter **exatamente 4 litros** no recipiente de 5 litros.

Como representar esse problema para que um algoritmo consiga buscar a solução?



Uma possibilidade:

(x, y) Exemplos:

$(0, 0)$

$(5, 0)$

$(0, 3)$

$(2, 3)$

$(4, 0)$

onde:

x = quantidade no recipiente de 5 L

y = quantidade no recipiente de 3 L

Pergunta:

Quais valores de (x, y) representam estados válidos?



Estado inicial:

$(0, 0)$ Teste de objetivo:

$x == 4$



Passo 2 — Estado inicial e objetivo

Portanto:

$(4, 0) \rightarrow$ solução

$(4, 1) \rightarrow$ solução

$(4, 2) \rightarrow$ solução

$(4, 3) \rightarrow$ solução



Passo 2 — Estado inicial e objetivo

Não precisamos necessariamente determinar antecipadamente qual deles será encontrado.



Passo 3 — Quais são os operadores?

A partir de um estado (x, y) podemos:

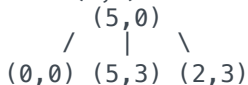
- 1 encher o recipiente de 5 L;
- 2 encher o recipiente de 3 L;
- 3 esvaziar o recipiente de 5 L;
- 4 esvaziar o recipiente de 3 L;
- 5 transferir 5 L $\rightarrow$ 3 L;
- 6 transferir 3 L $\rightarrow$ 5 L.

Essas regras definem como navegar pelo espaço.

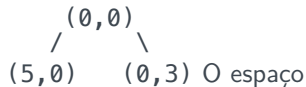


A partir de:

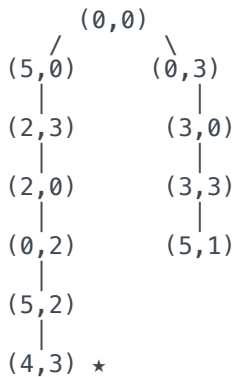
$(0,0)$ A partir de $(5,0)$:



podemos gerar:



O espaço começa a surgir a partir das **regras do problema**.





★ satisfaz:

$x == 4$ Logo, encontramos um estado objetivo.



Encontrar o estado $(4,3)$ não basta.



Precisamos recuperar o caminho:

$(0, 0)$



$(5, 0)$



$(2, 3)$



$(2, 0)$



$(0, 2)$



$(5, 2)$



$(4, 3)$



Mas qual é a solução?

Cada transição corresponde a uma ação.

A solução é o caminho entre o estado inicial e um estado objetivo.



Antes de escolher BFS, DFS ou outro algoritmo, responda:

- 1 Qual é a **solução buscada**?
- 2 Qual é o **estado inicial**?
- 3 Quais são as **ações possíveis**?
- 4 Existem estados inválidos?
- 5 Como gerar os estados sucessores?

- 1 Qual é o **problema**?
- 2 O que representa um **estado**?
- 3 Como identificamos um **estado objetivo**?
- 4 Qual é a **função de transição**?
- 5 Existem custos?

Somente depois escolha o algoritmo.



Um erro comum é começar perguntando:

Uma pergunta anterior é:

Representação inadequada



Espaço inadequado



Busca inadequada

"Qual algoritmo devo usar?"

"Como o problema será representado como um espaço no qual seja possível realizar uma busca?"



O labirinto é apenas um exemplo.
O mesmo raciocínio pode representar:



Rotas
cidade → cidade
Jogos
configuração → configuração
Planejamento
situação → situação
Dependências
componente → componente
Código
artefato → artefato relacionado
Testes
configuração → configuração



O domínio muda.

A estrutura fundamental permanece:

ESTADOS + TRANSIÇÕES + OBJETIVO

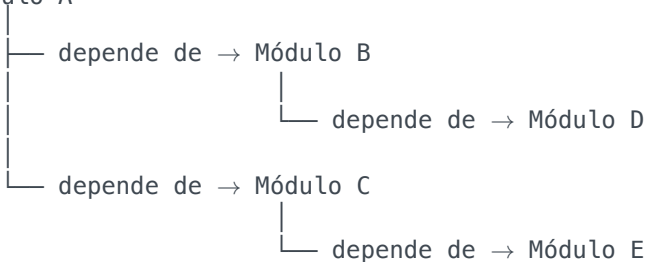


Em Engenharia de Software podemos perguntar:
Qual módulo está relacionado a uma falha?



Um espaço pode ser construído com:

Módulo A

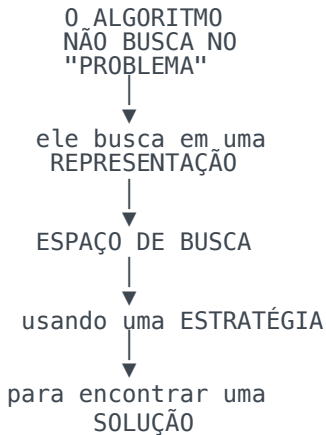




Agora podemos **buscar no grafo de dependências**.
O princípio é o mesmo utilizado no labirinto.



A ideia que deve permanecer





Para resolver um problema por busca:



1. compreender o problema
↓
2. definir a solução buscada
↓
3. representar os estados
↓
4. definir ações e transições
↓
5. caracterizar o espaço de busca
↓
6. escolher a estratégia de busca
↓
7. explorar o espaço
↓
8. reconhecer o objetivo
↓
9. reconstruir a solução



Agora podemos estudar os algoritmos sabendo **o que eles realmente fazem:**

- DFS;
- Greedy Search;
- Hill Climbing;
- Algoritmos Genéticos.
- BFS;
- Uniform Cost Search;
- A*;
- Simulated Annealing;

Cada estratégia oferece uma maneira diferente de explorar um espaço de possibilidades.



Até agora vimos algoritmos que exploram o espaço de busca de forma sistemática.

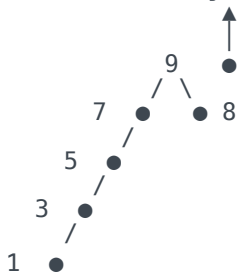
O **Hill Climbing** segue uma ideia diferente:

A partir do estado atual, escolha o vizinho que parece representar a maior melhoria.

Ele utiliza uma **função de avaliação** para decidir para onde seguir.



Imagine que cada estado possui uma "altura":
objetivo?





O algoritmo começa em algum ponto e pergunta:

Qual vizinho me leva para uma posição melhor?

Então ele segue sempre "subindo".

Daí o nome:

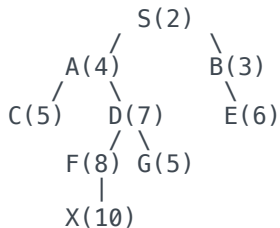
Hill Climbing

=

Subida da Encosta



Considere:





Os números representam o valor da função de avaliação.



Se queremos **maximizar**:

S(2)



A(4)



D(7)



F(8)



X(10)



Hill Climbing no espaço de busca

O algoritmo segue, a cada passo, para o melhor vizinho disponível.



Para usar Hill Climbing precisamos responder:
Podemos definir:

$f(\text{estado}) = \text{qualidade do estado}$

Dependendo do problema, queremos:

MAXIMIZAR $f(n)$

Como saber se um estado é melhor que outro?

ou:

$h(\text{estado}) = \text{estimativa de proximidade do objetivo}$

ou:

MINIMIZAR $h(n)$



Considere:



Podemos utilizar como avaliação a distância Manhattan:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

Quanto **menor** $h(n)$, mais próximo o estado parece estar do objetivo.



Suponha que o estado atual seja $(2,2)$ e o objetivo $(2,4)$:

```
vizinho esquerda → h = 3  
vizinho direita  → h = 1  
vizinho cima     → bloqueado  
vizinho baixo    → h = 3
```

Hill Climbing escolhe:

direita

porque é o movimento que mais melhora a avaliação.

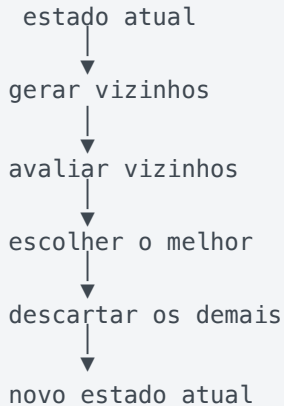


BFS pode manter:

```
fronteira = vários estados
```

DFS também mantém caminhos alternativos na pilha.

No Hill Climbing simples:

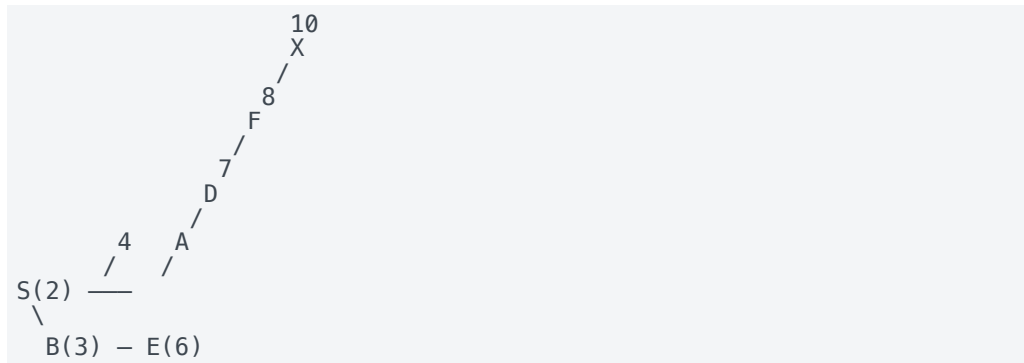




Ele concentra a exploração em uma **região local do espaço**.



Considere:





Hill Climbing pode explorar apenas:

$S \rightarrow A \rightarrow D \rightarrow F \rightarrow X$

Os outros estados podem nunca ser visitados.

Isso reduz a exploração, mas também cria riscos.



```
atual = estado_inicial
while True:
    if objetivo(atual):
        return atual
    vizinhos = gerar_sucessores(atual)
    melhor = melhor_estado(vizinhos)
    if avaliacao(melhor) <= avaliacao(atual):
        return atual
    atual = melhor
```



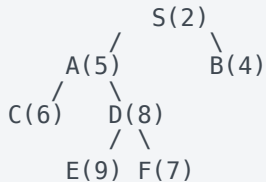
Para um problema de maximização.

A ideia central está em:

```
melhor_estado(vizinhos)
```



Considere:





Passo 1

```
Atual = S(2)  
Vizinhos = A(5), B(4)  
Escolha = A(5)
```



Passo 2

```
Atual = A(5)  
Vizinhos = C(6), D(8)  
Escolha = D(8)
```



Passo 3

```
Atual = D(8)  
Vizinhos = E(9), F(7)  
Escolha = E(9)
```



Suponha que o espaço tenha milhares de estados.

Hill Climbing pode explorar apenas:

```
S → D → K → P → X
```

Então temos:

```
menos memória  
+  
menos estados visitados  
+  
busca direcionada
```

BFS pode explorar:

```
S → A → B → C → D → E → F →  
G → ...
```

se a função de avaliação indicar que esse caminho é sempre melhor.

Mas existe uma contrapartida.



Imagine:





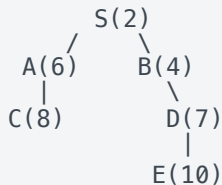
O algoritmo sobe até:

máximo local

e pode concluir:

"Não existe vizinho melhor."

Mesmo existindo uma solução global melhor em outra região do espaço.



Começando em S:

$S(2) \rightarrow A(6) \rightarrow C(8)$



Em C:

nenhum vizinho melhor

Então o algoritmo para em:

$C(8)$

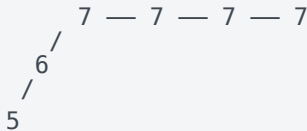
Mas existe:

$E(10)$

em outra região do espaço.



Um platô ocorre quando vários estados possuem a mesma avaliação:



O algoritmo pergunta:

Qual vizinho é melhor?

Mas encontra:

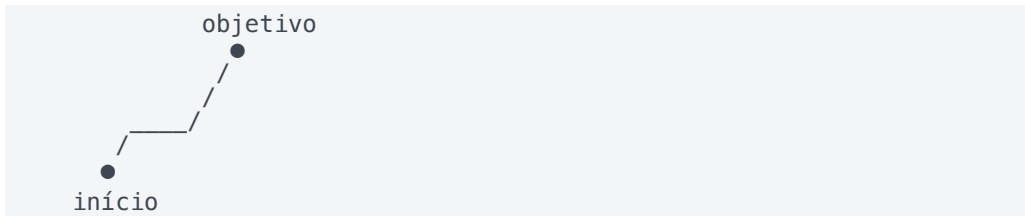
7, 7, 7...

Nenhum oferece melhoria clara.

A busca pode parar ou caminhar sem direção.



Imagine que o caminho de melhoria exige movimentos que isoladamente parecem ruins:





No espaço discreto, pode acontecer:

```
Atual = 7  
Vizinhos:  
6  
6  
7
```

Não existe melhoria imediata, embora uma sequência indireta pudesse levar a uma solução melhor.



Essa é uma diferença fundamental.

BFS

- completa em espaços finitos apropriados
- encontra caminho mínimo quando custos são uniformes

Hill Climbing

- busca local
- depende da função de avaliação
- pode parar antes do ótimo global

Portanto:

Hill Climbing troca garantias de busca por eficiência e direcionamento.



ESPAÇO DE BUSCA



BFS



Explora:

S, A, B, C, D, E, F, G, H...



DFS

```
Explora:  
S → A → D → ...
```

Hill Climbing

```
Explora:  
S → melhor vizinho  
    → melhor vizinho  
    → melhor vizinho...
```

O Hill Climbing tenta **reduzir drasticamente a região explorada**.



Estratégia	Como escolhe o próximo estado?	Memória	Orientação
BFS	mais antigo da fronteira	alta	não orientada
DFS	mais recente da fronteira	baixa	não orientada
Hill Climbing	melhor vizinho local	muito baixa	função de avaliação

Hill Climbing normalmente mantém:

estado atual
+
vizinhos atuais

em vez de armazenar uma grande fronteira.



O sucesso do Hill Climbing depende fortemente de:

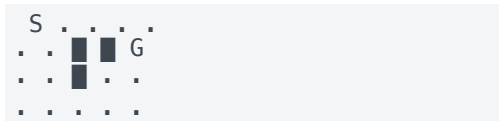
QUALIDADE DA REPRESENTAÇÃO
+
QUALIDADE DA FUNÇÃO DE AVALIAÇÃO

Uma função ruim pode levar o algoritmo para uma região ruim do espaço.

Uma função informativa pode tornar a busca extremamente eficiente.



Considere:



Hill Climbing simples pode ter dificuldade nesse cenário.

A distância até G pode indicar:

"vá para a direita"

Mas existe uma parede.

O caminho correto pode exigir primeiro:

afastar-se do objetivo

para depois se aproximar.



Algumas variações incluem:

Hill Climbing estocástico

Escolhe aleatoriamente entre movimentos que melhoram.

First-choice Hill Climbing

Escolhe o primeiro vizinho encontrado que melhora a solução.

Sideways Moves

Permite alguns movimentos com o mesmo valor.

Random Restart

Executa Hill Climbing várias vezes a partir de diferentes estados iniciais.



Uma forma de escapar de máximos locais:

A ideia é repetir a busca a partir de diferentes pontos do espaço.

```
Execução 1:  
S1 → máximo local  
Execução 2:  
S2 → máximo local  
Execução 3:  
S3 → ótimo global
```



Aqui aparece uma diferença conceitual importante.

Em problemas de otimização:

*cada estado pode representar uma **solução candidata completa**.*



Por exemplo:

```
Estado = uma possível escala de professores  
Estado = uma possível sequência de tarefas  
Estado = uma possível configuração de software  
Estado = uma possível arquitetura  
Estado = uma possível seleção de casos de teste
```

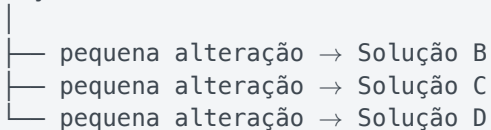


O Hill Climbing modifica uma solução candidata para produzir soluções vizinhas.



Imagine um problema de configuração:

Solução A



Cada pequena alteração define uma **vizinhança**.



O Hill Climbing procura:

```
solução atual
  ↓
melhor solução vizinha
  ↓
melhor solução vizinha
  ↓
...
```



Suponha que queremos priorizar uma suíte de testes.

Um estado pode ser:

```
[TC3, TC1, TC5, TC2, TC4]
```

Uma solução vizinha:

```
trocar duas posições
```

gera:

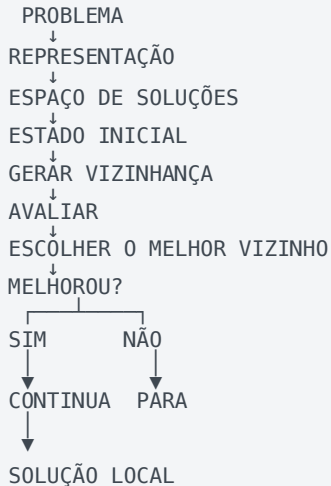
```
[TC3, TC5, TC1, TC2, TC4]
```



A função de avaliação pode considerar:

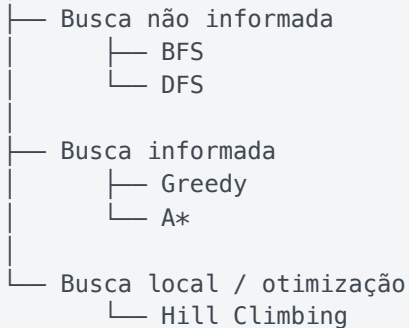
```
cobertura antecipada  
+  
tempo de execução  
+  
prioridade de falhas
```

Hill Climbing busca uma configuração melhor da suíte.





ALGORITMOS DE BUSCA





Hill Climbing não busca necessariamente um caminho.

Frequentemente busca:

uma configuração melhor dentro de um espaço de soluções candidatas.



```
BFS / DFS  
Estado inicial  
  ↓  
explorar caminhos  
  ↓  
encontrar objetivo  
  ↓  
reconstruir caminho
```



```
Hill Climbing
Solução candidata inicial
↓
gerar vizinhos
↓
avaliar vizinhos
↓
escolher melhoria
↓
repetir
↓
solução local
```



*A diferença essencial é que Hill Climbing realiza **busca local orientada por qualidade**, em vez de explorar sistematicamente o espaço inteiro.*